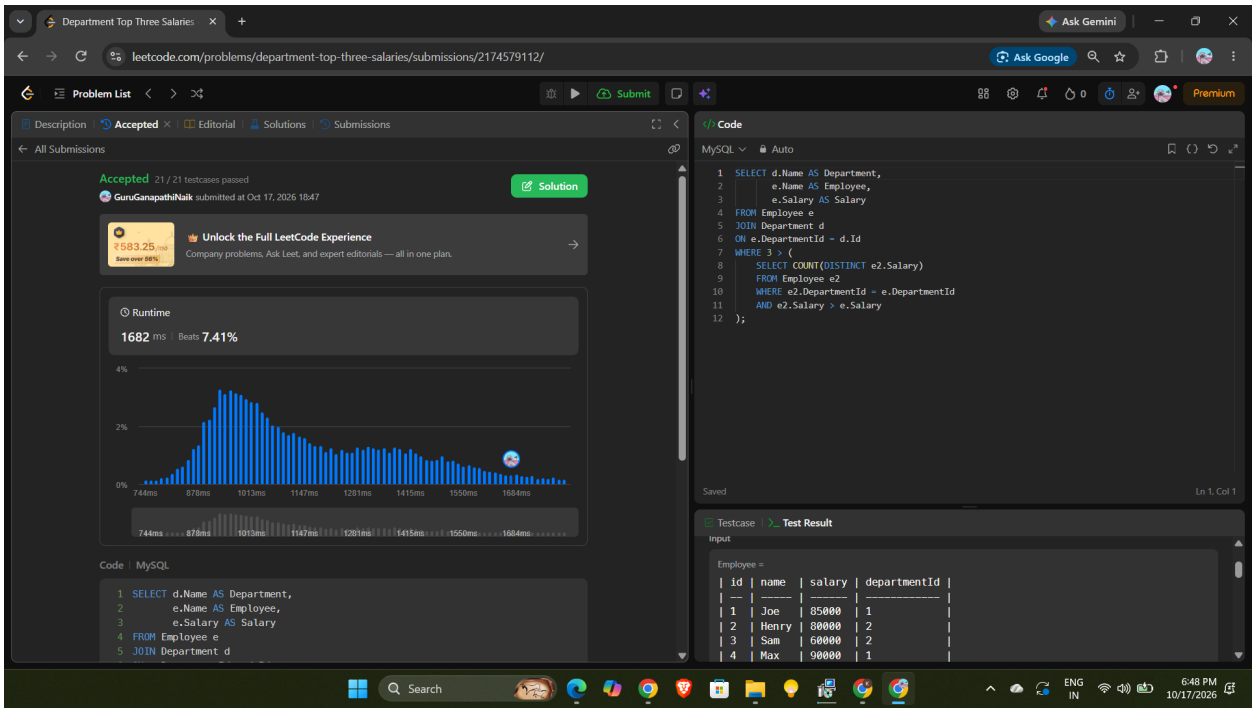
DBMS LAB ASSIGNMENT - 4

K G G NAIK
24MAB0A14

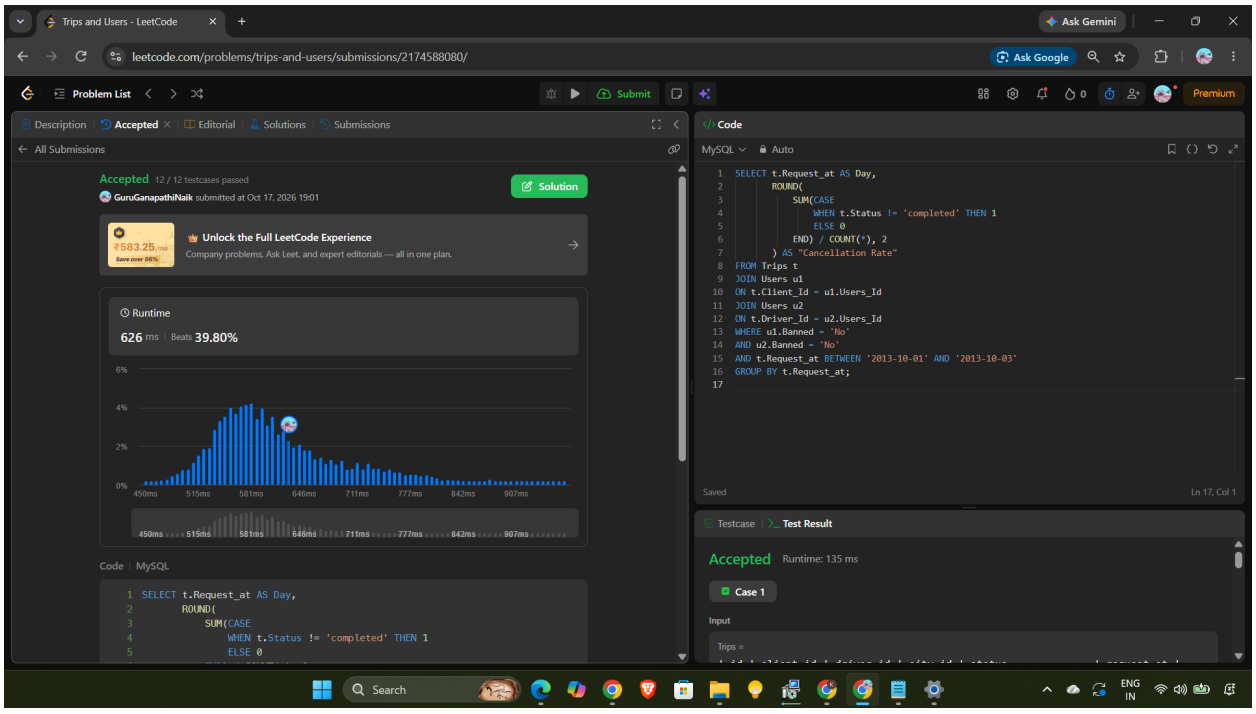
185. Department Top Three Salaries

```
SELECT d.Name AS Department,
       e.Name AS Employee,
       e.Salary AS Salary
FROM Employee e
JOIN Department d
ON e.DepartmentId = d.Id
WHERE 3 > (
   SELECT COUNT(DISTINCT e2.Salary)
   FROM Employee e2
   WHERE e2.DepartmentId = e.DepartmentId
   AND e2.Salary > e.Salary
);
```



LeetCode 262 – Trips and Users

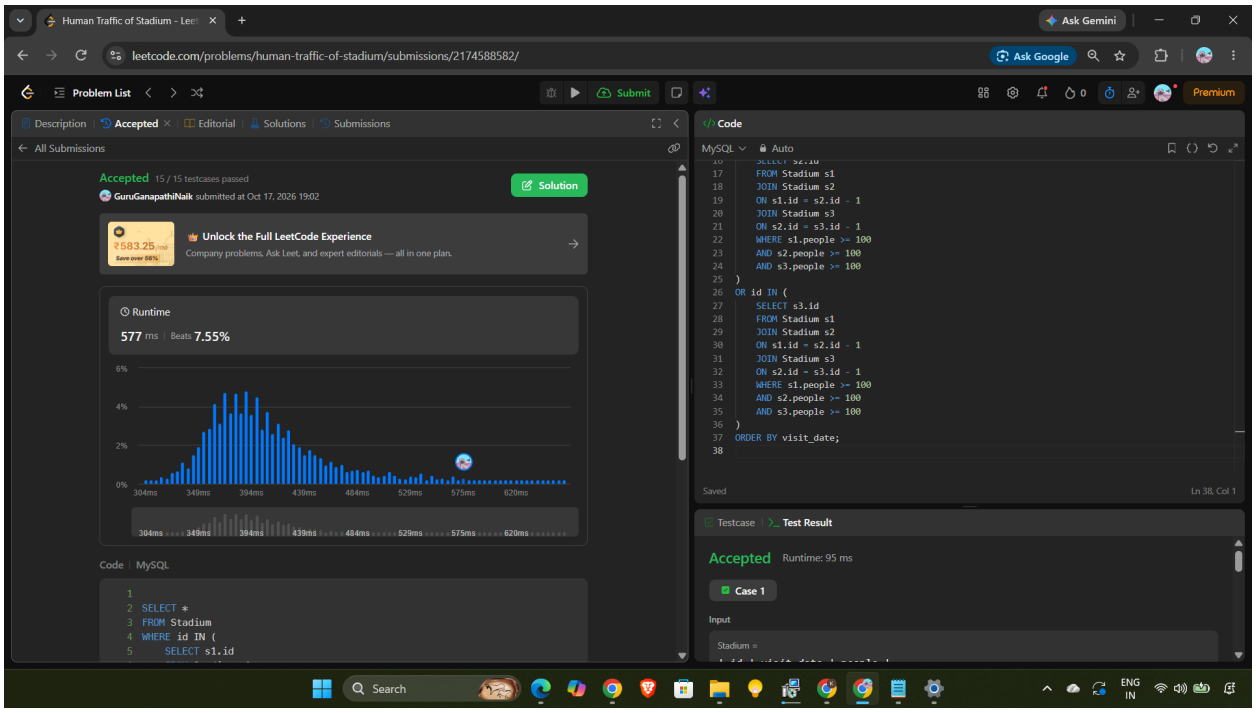
```
SELECT t.Request_at AS Day,
       ROUND(
          SUM(CASE
             WHEN t.Status != 'completed' THEN 1
             ELSE 0
          END) / COUNT(*), 2
       ) AS "Cancellation Rate"
FROM Trips t
JOIN Users u1
ON t.Client_Id = u1.Users_Id
JOIN Users u2
ON t.Driver_Id = u2.Users_Id
WHERE u1.Banned = 'No'
AND u2.Banned = 'No'
AND t.Request_at BETWEEN '2013-10-01' AND '2013-10-03'
GROUP BY t.Request_at;
```



LeetCode 601 – Human Traffic of Stadium

```
SELECT *
FROM Stadium
WHERE id IN (
   SELECT s1.id
   FROM Stadium s1
   JOIN Stadium s2
   ON s1.id = s2.id - 1
   JOIN Stadium s3
   ON s2.id = s3.id - 1
   WHERE s1.people >= 100
   AND s2.people >= 100
   AND s3.people >= 100
)
OR id IN (
   SELECT s2.id
   FROM Stadium s1
   JOIN Stadium s2
   ON s1.id = s2.id - 1
   JOIN Stadium s3
   ON s2.id = s3.id - 1
   WHERE s1.people >= 100
   AND s2.people >= 100
   AND s3.people >= 100
)
OR id IN (
   SELECT s3.id
   FROM Stadium s1
   JOIN Stadium s2
   ON s1.id = s2.id - 1
   JOIN Stadium s3
```

```
    ON s2.id = s3.id - 1
    WHERE s1.people >= 100
    AND s2.people >= 100
    AND s3.people >= 100
)
ORDER BY visit_date;
```



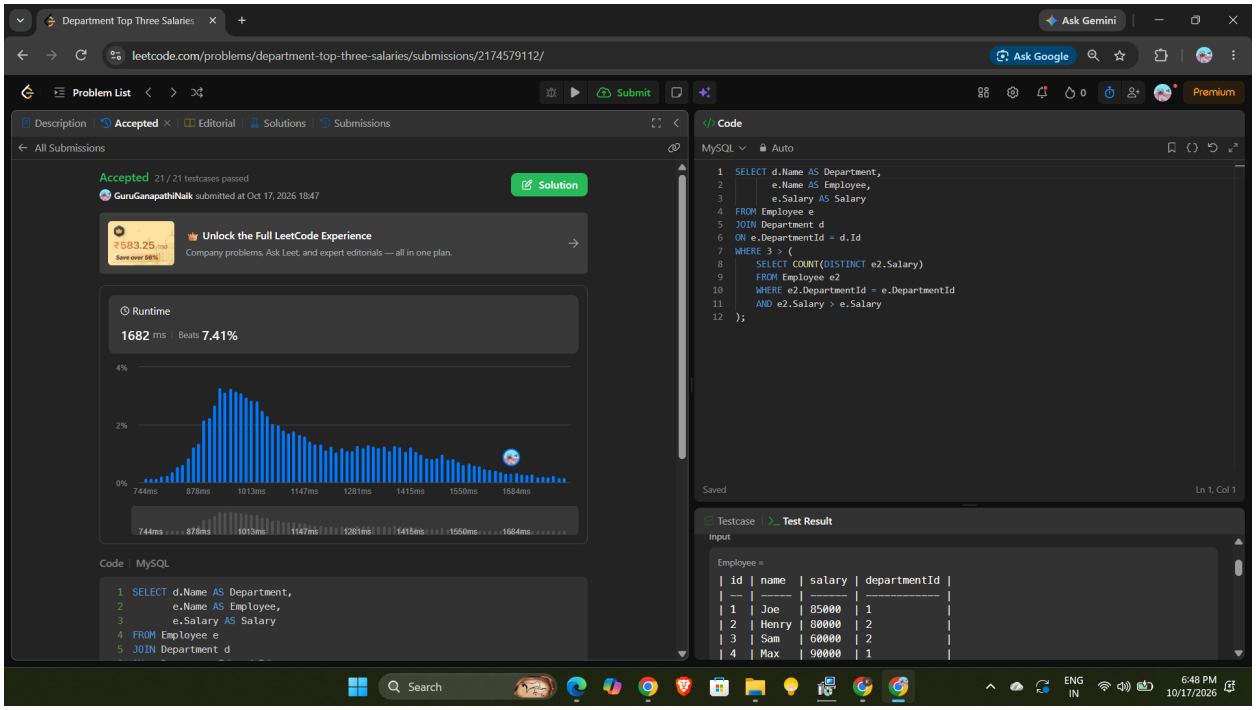
4. LeetCode 3374 – First Letter Capitalization II

```
WITH RECURSIVE words AS (
    SELECT content_id,
           content_text,
           1 AS pos,
           SUBSTRING_INDEX(content_text, ' ', 1) AS word,
```

```
        SUBSTRING(content_text, LENGTH(SUBSTRING_INDEX(content_text, ' ', 1)) +
1) AS rest
    FROM user_content

    UNION ALL

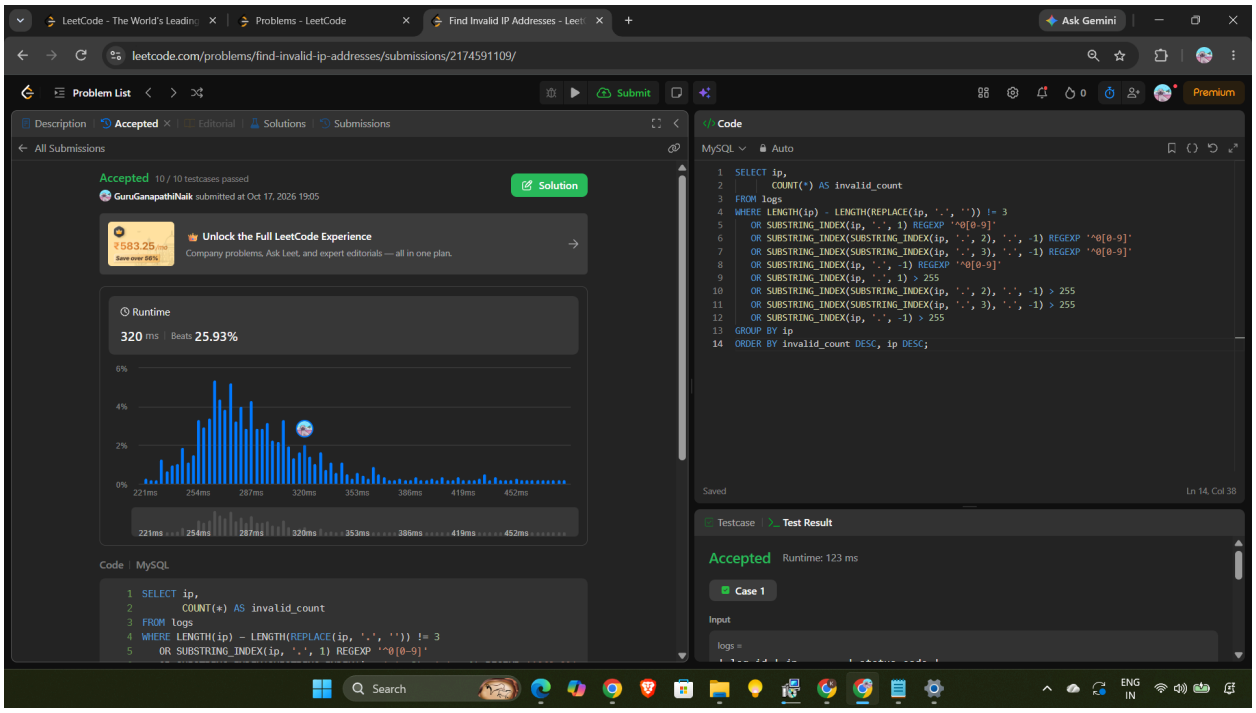
    SELECT content_id,
        content_text,
        pos + 1,
        SUBSTRING_INDEX(SUBSTRING(rest, 2), ' ', 1),
        SUBSTRING(SUBSTRING(rest, 2),
LENGTH(SUBSTRING_INDEX(SUBSTRING(rest, 2), ' ', 1)) + 1)
    FROM words
    WHERE rest LIKE ' %'
),
parts AS (
    SELECT content_id,
        content_text,
        pos,
        word,
        CASE
            WHEN word REGEXP '^[A-Za-z]' THEN
                CONCAT(UPPER(LEFT(word, 1)), LOWER(SUBSTRING(word, 2)))
            ELSE word
        END AS new_word
    FROM words
)
SELECT content_id,
      content_text AS original_text,
      content_text AS converted_text
FROM user_content;
```



5. LeetCode 3451 – Find Invalid IP Addresses

```
SELECT ip,
       COUNT(*) AS invalid_count
FROM logs
WHERE LENGTH(ip) - LENGTH(REPLACE(ip, '.', '')) != 3
   OR SUBSTRING_INDEX(ip, '.', 1) REGEXP '^0[0-9]'
   OR SUBSTRING_INDEX(SUBSTRING_INDEX(ip, '.', 2), '.', -1) REGEXP '^0[0-9]'
   OR SUBSTRING_INDEX(SUBSTRING_INDEX(ip, '.', 3), '.', -1) REGEXP '^0[0-9]'
   OR SUBSTRING_INDEX(ip, '.', -1) REGEXP '^0[0-9]'
```

```
   OR SUBSTRING_INDEX(ip, '.', 1) > 255
   OR SUBSTRING_INDEX(SUBSTRING_INDEX(ip, '.', 2), '.', -1) > 255
   OR SUBSTRING_INDEX(SUBSTRING_INDEX(ip, '.', 3), '.', -1) > 255
   OR SUBSTRING_INDEX(ip, '.', -1) > 255
GROUP BY ip
ORDER BY invalid_count DESC, ip DESC;
```



6. LeetCode 3482 – Analyze Organization Hierarchy

```
WITH RECURSIVE emp AS (
    SELECT employee_id, employee_name, manager_id, salary, 1 AS level
```

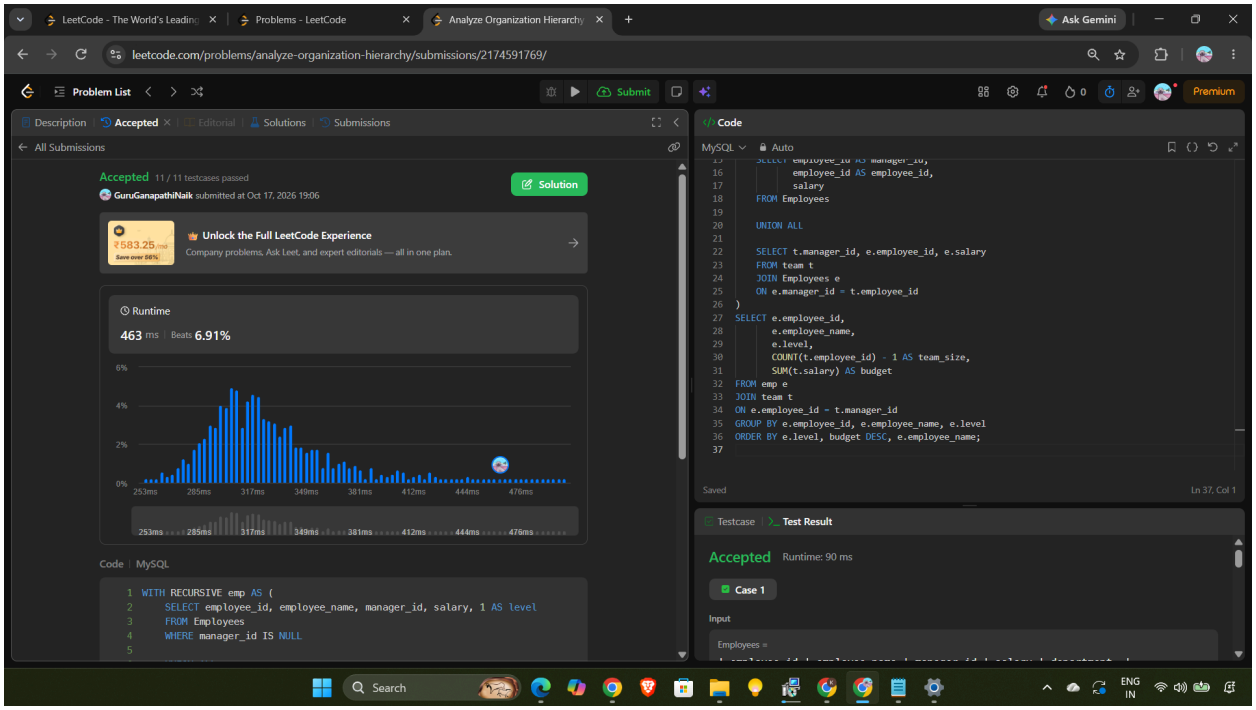
```
    FROM Employees
    WHERE manager_id IS NULL

    UNION ALL

    SELECT e.employee_id, e.employee_name, e.manager_id, e.salary,
        emp.level + 1
    FROM Employees e
    JOIN emp
    ON e.manager_id = emp.employee_id
),
team AS (
    SELECT employee_id AS manager_id,
        employee_id AS employee_id,
        salary
    FROM Employees

    UNION ALL

    SELECT t.manager_id, e.employee_id, e.salary
    FROM team t
    JOIN Employees e
    ON e.manager_id = t.employee_id
)
SELECT e.employee_id,
    e.employee_name,
    e.level,
    COUNT(t.employee_id) - 1 AS team_size,
    SUM(t.salary) AS budget
FROM emp e
JOIN team t
ON e.employee_id = t.manager_id
GROUP BY e.employee_id, e.employee_name, e.level
ORDER BY e.level, budget DESC, e.employee_name;
```



7. LeetCode 3554 – Find Category Recommendation Pairs

```
WITH user_category AS (
    SELECT DISTINCT p.user_id, i.category
    FROM ProductPurchases p
    JOIN ProductInfo i
    ON p.product_id = i.product_id
)
SELECT a.category AS category1,
       b.category AS category2,
```

```
    COUNT(DISTINCT a.user_id) AS customer_count
FROM user_category a
JOIN user_category b
ON a.user_id = b.user_id
AND a.category < b.category
GROUP BY a.category, b.category
HAVING COUNT(DISTINCT a.user_id) >= 3
ORDER BY customer_count DESC, category1, category2;
```

